订处

得分：

数据科学期末报告

题　　目：Transformer 学习方法及其在三角网格神经渲染中的应用

院　　系：计算机科学与技术

班　　级：

学生姓名：

学　　号：

完成日期：2026 年　　月　　日

大连理工大学计算机科学与技术学院

一 研究问题与学习任务

神经渲染将图像生成的一部分过程表示为可学习映射。本文选择 Transformer 作为学习方法，以 RenderFormer 为应用实例，讨论三角网格如何转化为网络输入、注意力如何汇聚场景信息，以及参考图如何构成监督信号。实验包含官方示例复现、Bunny 与简化 Sponza 的解码器微调，以及论文所述相机和光源边界条件测试。

传统路径追踪根据几何、材质与光源采样光路，逐步估计像素辐射亮度。RenderFormer 则从场景三角形与相机直接预测图像。两者的输入描述可以对应，但计算机制不同：前者显式估计光传输积分，后者通过训练样本学习场景到图像的统计关系。它的优势和误差都需要通过受控对照判断，不能仅凭视觉上合理的图片推断其恢复了完整物理过程。

本任务可写为监督学习问题。输入 x 包含三角形、法线、反射与发光参数和相机；目标 y 是同一场景的参考图；网络 fθ 将 x 映射为预测图。学习过程通过损失比较预测与参考，再用梯度更新参数。推理时只提供 x，不需要参考图。训练标签由 Cycles 产生，因此材质含义、相机、坐标约定和颜色空间必须在两条渲染路径之间对齐。

$$\hat{y} = f\theta(x), \quad \theta* = arg\ min\theta\ \Sigma_i\ \ell(f\theta(x_i), y_i)$$

数据的统计单位也影响结论。如果同一几何和材质配置的相近视角分别进入训练与测试，模型可能利用高度相似的内容获得较低误差。本文的微调按完整粗糙度配置划分集合，但几何与相机范围仍共享，因此研究范围限定为固定场景族内的材质插值。论文完整训练与本次小规模适配的参数范围和数据规模分别说明。

实验先验证预训练模型与参考渲染的基本流程，再在固定场景中考察材质变化，最后主动改变训练分布中的关键条件。这样的安排使报告能够同时回答三个问题：模型是否能够正常运行、有限数据能否改变输出，以及哪些输入变化仍会引起明显失败。

二 Transformer 的表示与注意力计算

Transformer 以 token 序列为输入，每个 token 是描述一个元素的向量。文本任务中的元素通常是词或子词，RenderFormer 中的元素主要对应场景三角形。三角形不仅有索引，还包含几何与材质信息；模型需要区分空间位置相近但朝向不同的表面，也要区分相同位置处具有不同反射属性的表面。

设输入表示为 X，三个可学习线性映射分别得到查询 Q、键 K 和值 V。查询表示当前元素需要读取的信息，键用于匹配查询，值提供被汇聚的内容。单头缩放点积注意力先计算 Q 与 K 之间的相关程度，再以 softmax 得到每行和为 1 的权重，最后对 V 加权求和。

$$Q = XWQ, \quad K = XWK, \quad V = XWV$$

$$Attention(Q,K,V) = softmax(QK^{\mathrm{T}} / \sqrt{d_{\mathrm{k}}})V$$

例如三个缩放后得分为 0、1、2，softmax 权重约为 0.090、0.245、0.665。若对应值为 1、2、4，加权结果约为 3.24。模型通过训练学习投影矩阵，而不是人为指定某个表面应该读取另一个表面的多少光照。注意力权重本身也不等于物理可见性或能量传递系数。

自注意力的查询、键和值来自同一组表示，适合三角形之间交换信息；交叉注意力的查询来自另一组输入，例如相机射线束，而键和值来自场景表示。多头结构在不同投影空间并行计算关系，合并后交由前馈网络变换。残差连接保留原有表示，归一化控制特征尺度，使深层网络更易优化。

空间位置不能仅由数组顺序表达。三角形文件中的排列顺序通常没有几何意义，RenderFormer 通过空间旋转位置编码向注意力计算引入相对位置关系。位置编码与法线特征共同帮助模型区分表面的朝向和距离。密集注意力仍需要处理近似 N^2 的成对关系，因此三角形数量对显存和运行成本有直接影响。

学习时，损失的梯度沿输出头、注意力和输入嵌入传播，逐步调整投影与非线性变换。本文微调冻结 Transformer，仅更新 DPT 输出头，因而可以比较固定场景特征是否足以通过较小的参数适配降低误差；它不能检验重新学习场景注意力本身会带来多少收益。

三 RenderFormer 的两阶段网络

RenderFormer 的第一阶段构建与视角无关的场景表示。三角形材质和法线经过编码得到 token，三角形坐标通过空间 RoPE 参与注意力。场景 Transformer 使不同表面之间交换特征，得到包含全局场景条件的表示。第二阶段引入相机位置、方向与视场角，生成射线束查询，利用交叉注意力读取场景信息，再由 DPT 解码为图像。

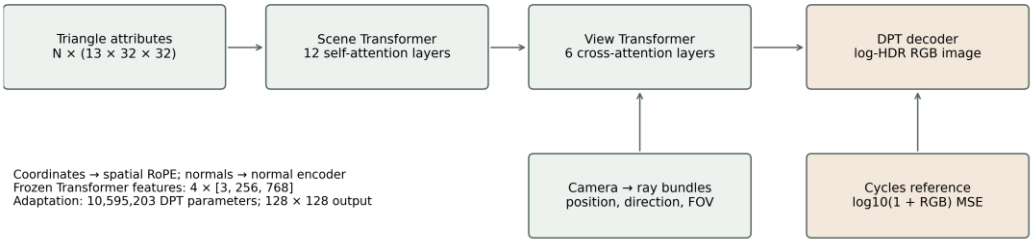


图 1　官方 base 网络结构及本次 DPT 微调位置

原论文的基础材质说明包括漫反射 RGB、镜面反射 RGB、粗糙度和发光 RGB。实际使用的发布 base 权重采用 13 通道、32×32 的三角形纹理编码，并包含法线相关通道。本文按代码接口准备数据，不将论文中的简化表示直接代替发布模型的输入。基础模型场景 Transformer 为 12 层，视角 Transformer 为 6 层，特征维度为 768。

场景转换输出 HDF5 文件，主要字段为 triangles、vn、texture、c2w 和 fov，分别保存顶点、顶点法线、材质纹理、相机变换和视场角。几何统一三角化，并在模型和 Cycles 参考中使用同一转换结果。Sponza 先作几何简化和材质近似，再进入这套共同输入流程。

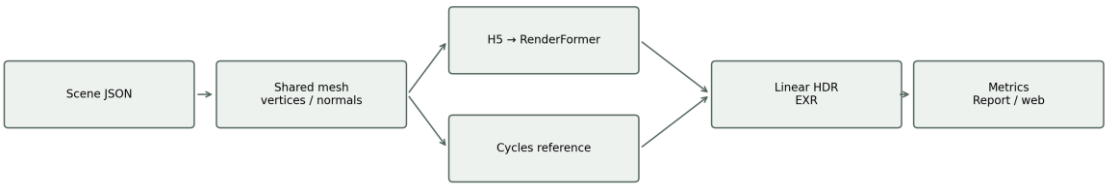


图 2　共享场景输入到模型预测和参考渲染的数据流程

HDR 像素的数值范围可能很大。发布模型采用对数亮度表示，输出经逆变换还原到线性 HDR。显示图另作固定曝光、Reinhard 色调映射和 sRGB 变换。原始线性结果与显示图用途不同：前者用于保留亮度信息和计算对数损失，后者用于观察与显示域结构评价。

实现上，官方推理封装关闭梯度。微调时缓存视角 Transformer 四个层容器中的特征，保持原始空间形状，并单独训练 DPT。128 像素、三视图配置的各层缓存形状为[3,256,768]。这样避免每次更新重复执行冻结主干，同时保留与本轮参考图对应的射线与场景条件。

四 数据采集与训练设置

实验服务器采用 RTX 2080 Ti，单卡显存 11 GB，Python 3.10、PyTorch 2.5.1 与 CUDA 12.4。官方代码固定为 eec6a9b8d592303def45e458dd4ebd09d6572abc，基础权重为 microsoft/renderformer-v1-base，revision 为 8aa0009563789dbb6c7785a26aac5da2c9b6d117。注意力使用 PyTorch SDPA。

Bunny 采用官方康奈尔盒场景。Sponza 来自 Khronos 公开 glTF 样例，原始 262,267 面简化为 19,235 面，纹理转为平均颜色，透明和法线贴图未保留。简化后的几何与材质同时供模型和参考使用。为获得可见的室内照明，Sponza 使用场景内发光三角形；这一条件本身也超出论文的部分训练设置，是观察适配效果的重要背景。

集合	粗糙度配置	图像数
训练	0.15、0.50、0.90	18
验证	0.30	6
测试	0.70	6

每个场景每种配置含三个相近视图，共 30 帧。参考使用 Cycles、128×128 分辨率、512 spp。训练与测试的几何相同，视角范围重叠；上述划分隔离的是材质配置，而不是完整场景。报告中不将它称为严格的分布外测试。

冻结全部 Transformer 参数，仅更新 10,595,203 个 DPT 参数。损失取预测和参考在 log10(1+max(RGB,0))空间中的逐像素均方误差。优化器为 AdamW，学习率 10^{-5}，权重衰减 10^{-4}，梯度范数上限为 1，随机种子为 20261101，执行 30 轮，每轮 6 个配置更新。

$$L = (1 / 3HW)\,\Sigma_{\mathrm{p},c}\,[log_{10}(1 + \hat{y}_{\mathrm{p},c}) - log_{10}(1 + y_{\mathrm{p},c})]^2$$

初次 FP16 训练出现梯度溢出，未作为最终结果。正式结果改为 FP32 解码器，并从原始输出头开始。每轮用验证配置计算损失，保留验证误差最低的权重；本次最低值出现在第 30 轮。训练达到预设轮数时验证误差仍下降，因此这里只报告完成 30 轮，不把它解释为稳定收敛。

五 微调结果与误差分析

预测和参考统一负值截零、EV0 曝光、逐通道 Reinhard 与 sRGB 变换。PSNR 和 SSIM 在变换后的浮点 RGB 上计算，data_range=1；log HDR MSE 在对数线性数据上计算。表 1 为每个场景三个留出视图指标的算术平均，均在相同 128 像素输入、参考与显示条件下比较。

表 1　留出配置的预训练与微调结果

场景	PSNR / dB	SSIM
Bunny	15.46 → 20.53	0.604 → 0.693
Sponza	3.10 → 15.83	0.098 → 0.452

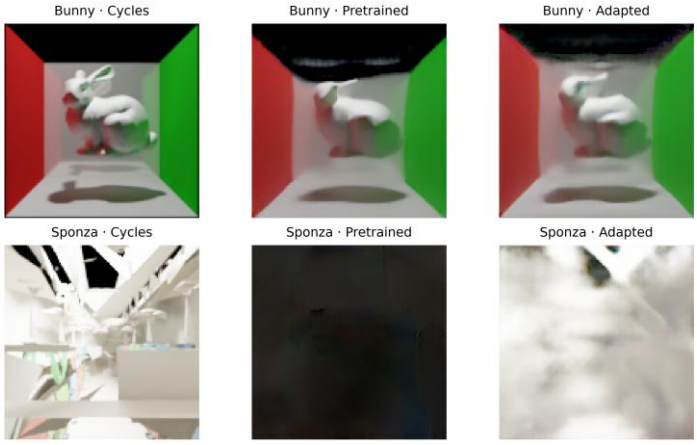


图 3　留出中间视图的参考、预训练与微调结果，128×128

Bunny 的轮廓与背景色渗透较预训练预测接近参考，但耳部和接触阴影仍有模糊。Sponza 的预训练图明显偏暗，适配后亮度和主要分区改善较大，不过细小建筑结构仍不能恢复。数值改善不能代替对细节的判断，尤其不能将这种低分辨率、固定建筑的适配推广为完整 Sponza 渲染能力。

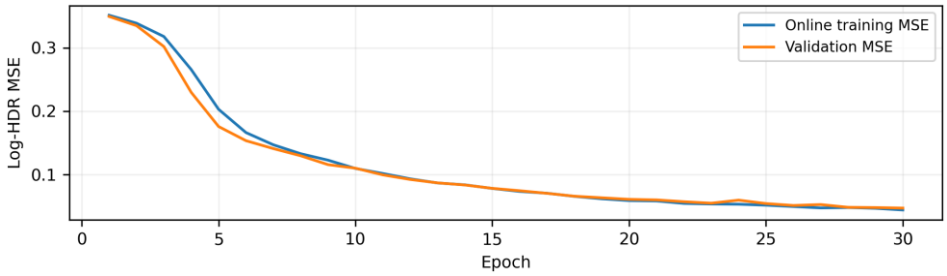


图 4　训练与验证损失；训练为轮内均值，验证为轮末测量

训练值和验证值采用不同统计时点，不能把两条曲线的间距直接当作严格的泛化差距。30 轮后训练对数 MSE 约 0.04429，验证约 0.04723。对两类场景而言，验证均值还掩盖了难度差异，应同时检查逐场景指标与图像。

六 论文边界条件的对照实验

论文指出模型对场景内相机、场景内光源、超出训练范围的光源尺寸和数量存在局限。本文从官方康奈尔盒出发，分别改变这些条件，使用 512×512 预测与 2048 spp Cycles 参考。各条件共用网格和材质转换规则，除被考察的相机或光源条件外保持配置不变。

条件	PSNR / dB	SSIM
基准	33.21	0.975
内部相机	12.60	0.810
内部光源	13.36	0.819
大面积光源	2.75	0.153
9 光源直接	15.32	0.729
9 光源分解相加	10.29	0.727

相机由外部移入场景后，预测出现明显亮度与结构误差；内部光源也使亮度分布偏离参考。大面积光源使用相同辐亮度，总发光功率随面积增大，此项考察超范围尺寸条件，并未控制总功率。它造成最明显的整图失真。不同条件的指标是各自对参考的误差，不应解释成一个共同目标上的模型排名。



图 5　内部相机条件下的参考、预测与误差，512×512

9 光源直接预测与分解预测使用同一 9 光源 Cycles 参考。分解预测分别只开启一个光源，得到 9 张线性 HDR 图后相加，最后统一做显示变换。本次分解的 PSNR 低于直接预测，因此没有复现论文中该修正方式带来的改善。神经预测并未严格满足光传输的线性关系，逐次近似误差也可能在相加时累积。

这些测试复现的是论文描述的条件类型，而不是逐像素复现原论文图像。复杂阴影、高阶反射以及高分辨率依赖没有在本轮独立测量，结论仅涉及实际运行的条件。完整对照图、固定色标误差图和逐条件指标保留在配套网页。

七 讨论与结论

实验表明，在预训练场景与视角特征固定时，约一千万个 DPT 参数仍可通过少量监督样本改变输出，并降低 Bunny 和简化 Sponza 材质留出配置的误差。改善主要体现在整体亮度和主要结构；微小几何、局部阴影和高光细节仍有限。这说明图像解码器具有适配能力，也表明冻结特征、低分辨率标签和有限训练配置会限制可恢复的信息。

边界实验则显示，注意力结构能够读取全局场景，不意味着输入条件变化后仍能保持物理正确。内部相机改变了可见几何与射线关系，内部光源改变了遮挡与照明条件；光源尺寸和数量变化还会改变 token 统计以及总能量。若要进一步改善这些情况，训练数据必须覆盖对应条件，并分别检查图像误差、几何轮廓与光照响应。

本次实验仍有三方面限制。第一，微调仅 30 帧，测试与训练共享几何和相近视角，不能评价新建筑的泛化。第二，Sponza 使用简化网格和平均颜色，不能代表原始贴图材质的难度。第三，512 spp 和 2048 spp 是固定参考预算，并非所有局部高光都完成了独立的采样收敛验证。因此对小幅指标差异应谨慎解释，较大结构错误则可结合对照图确认。

后续工作应优先增加场景与相机配置的独立性，扩大验证集合，并在相同分辨率下检查细节恢复。若继续优化训练，不应根据当前测试分数反复挑选权重；需要重新锁定评估配置。解码器适配还应在官方盒子和其他材质条件下进行回归检查，以观察受限训练是否损害原有能力。

通过本实验，Transformer 的学习过程可以落到具体数据上：将场景属性编码为 token，通过注意力交换与读取信息，再用参考图的损失修正参数。模型是否有效，应同时由数据划分、同条件指标、视觉误差和失败条件来说明。本文完成了可复现的基础推理、受限微调和四类光照相机边界测试，也保留了未达到预期的光源分解结果。

参考文献

[1] Zeng C, Dong Y, Peers P, et al. RenderFormer: Transformer-based Neural Rendering of Triangle Meshes with Global Illumination. SIGGRAPH Conference Papers, 2025. arXiv:2505.21925.

[2] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need. Advances in Neural Information Processing Systems, 2017.

[3] Microsoft. RenderFormer official implementation. github.com/microsoft/renderformer. 固定版本 eec6a9b8.

[4] Khronos Group. glTF Sample Assets: Sponza. github.com/KhronosGroup/glTF-Sample-Assets.

配套实验与逐项数据：www.kiiye9697.cn/renderformer-study/